

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is light green. They are positioned diagonally, with the blue one partially covering the green one.

# Projet: Implémentez un modèle de scoring



# Sommaire

1. Contexte et problématique
2. Notebook de référence & modélisation
3. Point CI
4. Conclusions



# Contexte et problématique

## Contexte

L'entreprise "**Prêt à dépenser**", spécialisée dans les crédits à la consommation pour des clients avec peu d'historique bancaire, souhaite développer un **outil de scoring crédit** fiable et transparent.

L'objectif est d'automatiser la **décision d'octroi de crédit** tout en **limitant les risques de défaut** et en **expliquant les décisions du modèle**.

## Problématique

Comment concevoir, évaluer et déployer un **modèle de scoring crédit robuste et explicable**, capable de **prédire la probabilité de remboursement d'un client**, tout en intégrant une **approche MLOps** pour assurer le **suivi, la reproductibilité et l'amélioration continue** du modèle ?

# Notebook de référence & modélisation: Données

Notebook base:

Les données qui vont nous intéresser sont dans la table 'application\_{train|test}.csv'

C'est la table principale, divisée en deux fichiers :

- *application\_train.csv* : contient la variable cible TARGET
- *application\_test.csv* : sans la variable TARGET

Chaque ligne représente un prêt accordé (une demande de crédit)

| DAYS_BIRTH | DAYS_EMPLOYED | DAYS_REGISTRATION | DAYS_ID_PUBLISH | OWN_CAR_AGE |
|------------|---------------|-------------------|-----------------|-------------|
| -9461      | -637          | -3648.0           | -2120           | NaN         |
| -16765     | -1188         | -1186.0           | -291            | NaN         |
| -19046     | -225          | -4260.0           | -2531           | 26.0        |
| -19005     | -3039         | -9833.0           | -2437           | NaN         |
| -19932     | -3038         | -4311.0           | -3458           | NaN         |

| CODE_GENDER | FLAG_OWN_CAR | FLAG_OWN_REALTY | CNT_CHILDREN | AMT_INCOME_TOTAL | AMT_CREDIT | AMT_ANNUITY |
|-------------|--------------|-----------------|--------------|------------------|------------|-------------|
| M           | N            | Y               | 0            | 202500.0         | 406597.5   | 24700.5     |
| F           | N            | N               | 0            | 270000.0         | 1293502.5  | 35698.5     |
| M           | Y            | Y               | 0            | 67500.0          | 135000.0   | 6750.0      |
| F           | N            | Y               | 0            | 135000.0         | 312682.5   | 29686.5     |
| M           | N            | Y               | 0            | 121500.0         | 513000.0   | 21865.5     |



# Notebook de référence & modélisation: Feature engineering

## Création de feature dites “*de domaine*”

*Création de quelques variables censées refléter des facteurs importants pour prédire si un client risque de ne pas rembourser son prêt. (inspiré d'un script tiers)*

- **CREDIT\_INCOME\_PERCENT** : pourcentage du montant du crédit par rapport au revenu du client
- **ANNUITY\_INCOME\_PERCENT** : pourcentage du montant de l'annuité du prêt par rapport au revenu du client
- **CREDIT\_TERM** : durée totale du remboursement en mois (l'annuité correspondant au montant mensuel à payer)
- **DAYS\_EMPLOYED\_PERCENT** : pourcentage du nombre de jours travaillés par rapport à l'âge du client



# Notebook de référence & modélisation: Feature engineering

## Création de feature dites “*polynomiales*”

*Ces features consistent à créer de nouvelles variables en combinant les puissances et interactions des variables existantes (ex:  $EXT\_SOURCE\_1 \times EXT\_SOURCE\_2^2$ ). Ces termes d'interaction permettent de capturer les effets combinés entre variables, pouvant révéler des relations non linéaires avec la cible.*

- **EXT\_SOURCE\_1 DAYS\_BIRTH**: produit des valeurs EXT\_SOURCE\_1 et DAYS\_BIRTH
- **EXT\_SOURCE\_2^2**: valeurs EXT\_SOURCE\_2 au carré
- ...



# Modélisation

- Création d'une fonction custom business score inspiré d'un scorer de churn

```
from sklearn.metrics import confusion_matrix, make_scorer

def custom_business_score(y_true, y_pred, cost_fp=5, cost_fn=1):
    """
    - y_true : vraies étiquettes
    - y_pred : prédictions binaires
    - cost_fp : coût d'un faux positif
    - cost_fn : coût d'un faux négatif
    """

    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

    total_cost = (fp * cost_fp) + (fn * cost_fn)

    return -total_cost # On retourne le négatif pour que "plus c'est grand, mieux c'est"

business_scorer = make_scorer(custom_business_score, greater_is_better=True)
```



# Modélisation

2 modèles: Régression Linéaire, Random Forest

3 datasets: dataset "de base", dataset base + feature polynomiales, dataset base + domain features

Pipeline:

- > Gestion du data leakage avec SMOTE (score: 0.48 -> 0.7)
- > Split train test des datasets
- > Entraîner le modèle avec grid\_cv afin de trouver les meilleurs hyperparametres
- > Log les metriques dans MLFlow
- > Recherche du meilleur seuil avec le dataset de test avec la métrique AUC calculée lors de la phase de validation



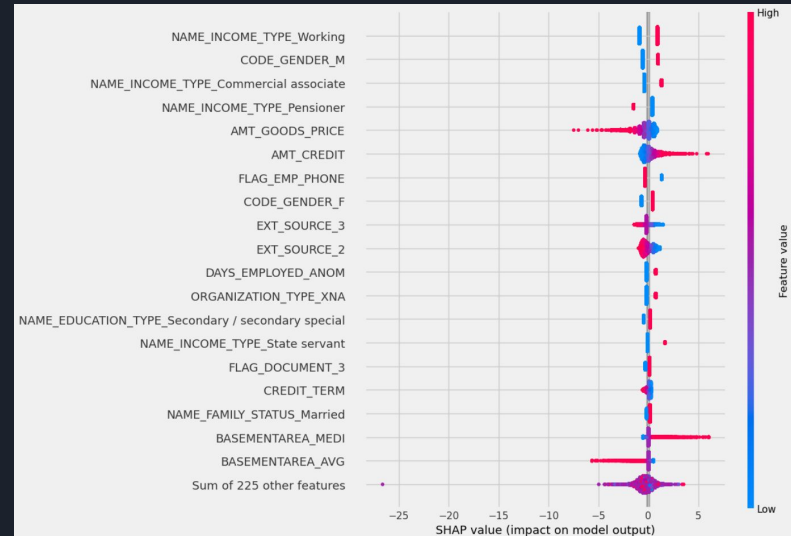
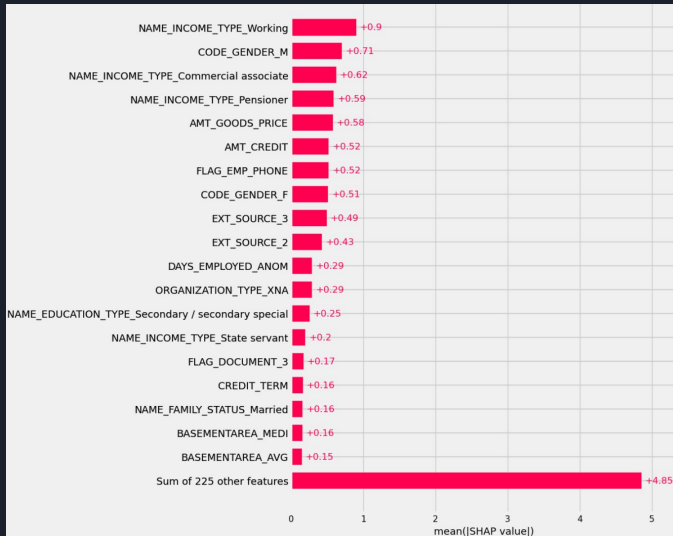


# Modélisation: résultats

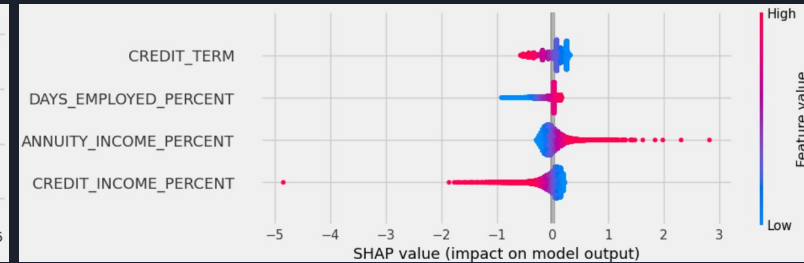
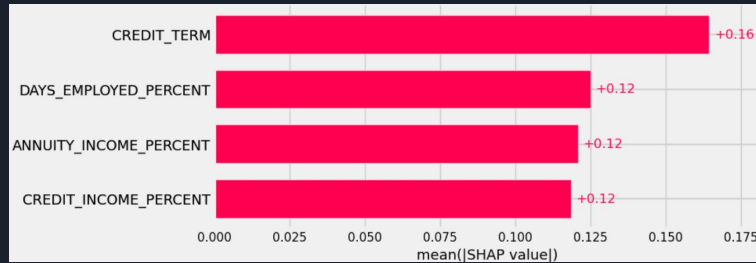
| AUC (validation)      | Modèle original du notebook | Dataset “de base” (baseline) | Dataset features domaine | Dataset feature polynomiales |
|-----------------------|-----------------------------|------------------------------|--------------------------|------------------------------|
| Régression logistique | 0.69                        | 0.75                         | 0.75                     | 0.41                         |
| RandomForest          | 0.67                        | 0.69                         | 0.69                     | 0.56                         |

# Modélisation: feature importance

Notre modèle le plus performant est Régression Logistique sur le dataset connaissance domaine



# Modélisation: feature importance sur les features domaine



-> Les domain features ont peu d'impact sur la prédiction



## Modélisation: interprétation des resultats du dataset “Features polynomiales”

Visiblement le notebook de base enlève certains colonnes du dataset baseline et ajoute les poly  
les colonnes retirées sont ['EXT\_SOURCE\_1', 'EXT\_SOURCE\_2', 'EXT\_SOURCE\_3',  
'DAYS\_BIRTH']



# Modélisation: data drift

| Dataset Drift                                                           |                       |                                   |
|-------------------------------------------------------------------------|-----------------------|-----------------------------------|
| Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5 |                       |                                   |
| 244<br>Columns                                                          | 32<br>Drifted Columns | 0.131<br>Share of Drifted Columns |

## Calcul du *data drift* avec “Evidently”

du drift a été identifié sur 32 colonnes

Le data drift dans Evidently est calculé en comparant la distribution des variables entre un jeu de référence (données historiques) et un jeu courant (nouvelles données) et estime si la distribution a significativement changé (Kolmogorov–Smirnov ou  $\chi^2$ )

Une bonne piste pour gérer le drift est de mettre en place un système de surveillance continue : si le score de drift dépasse un seuil défini (par exemple 0.5 ou 0.7 selon la métrique), on peut déclencher automatiquement un réentraînement du modèle avec des données récentes.

# Pipeline CI/CD

- Diagramme de la pipeline CI / CD



- Config du serveur render (ajouter les log du run build avec les tests)

## Start Command

Render runs this command to start your app with each deploy.

```
$ pip install pytest && pytest -v test_api.py || exit 1 uvicorn api:app --host 0.0.0.0 --port 8000
```

[Edit](#)

# Demo d'appel api





# Conclusions

- Récupération d'un kernel, exploration du feature engineering
  - Tests de modèles avec les différents datasets, évaluations et investigations des résultats
    - Amélioration de 21% avec redressement de la donnée
    - Amélioration de 5% avec le dataset connaissance domaine
    - Meilleur modèle 75% accuracy
  - Mise en place d'un environnement CI/CD avec déploiement d'une API
  - Test de l'API
- 
- Feature engineering relativement peu impactant, améliorations possibles
  - Changer de pipeline CI / CD pour une meilleure intégration des tests